

Antoine KAHN
Kelvin LEFORT

I - Introduction

Ce document présente le travail réalisé dans le cadre du projet du cours de Programmation Objet Avancée (POO) du M1 d'Informatique de l'Université Paris Cité. L'objectif était la réalisation, en C++, d'une version du jeu *Baba Is You*. Ce travail est disponible sur GitHub via le lien <https://github.com/dovinzo/baba-is-you>. Pour l'affichage graphique, nous avons utilisé la version 2.5.1 de SFML (<https://www.sfm1-dev.org/>).

Ce jeu a pour principale caractéristique d'intégrer les règles directement comme des éléments du jeu. Ainsi, la règle permettant de faire en sorte qu'un élément soit poussable, bloquant, etc. est un objet du jeu dynamique en plus d'être une entité sémantique. Notre implémentation propose un menu où l'utilisateur sélectionne le niveau désiré, puis le niveau se lance. Pour réaliser ce projet, nous avons cherché à proposer une implémentation qui respecte les différents principes de la POO, à savoir l'abstraction, l'encapsulation, la modularité et la hiérarchisation.

Dans la suite de ce rapport, nous nous intéresserons aux caractéristiques techniques de notre implémentation, ainsi qu'aux raisons qui nous ont poussés à sélectionner celles-ci. Après avoir décrit une vue globale de l'architecture de notre projet (II), nous nous intéresserons plus en détail à ses trois éléments constitutifs principaux, à savoir l'architecture de l'application (III), celle du menu (IV) et celle du niveau choisi (V). Enfin, la dernière partie sera consacrée aux développements possibles de notre implémentation : les problèmes connus et les améliorations possibles (VI).

II - Vue d'ensemble

Dans cette section, on explique le fonctionnement global de l'application. Les détails techniques seront apportés dans les prochaines sections.

On a décidé de mettre en place les 3 premiers niveaux de *Baba Is You*. L'utilisateur peut sélectionner un niveau à l'aide d'un menu. Il se trouve alors soit sur le menu, soit sur le niveau choisi.

Voici une explication des touches lors du **menu** :

Touche	Explication
Flèche vers le haut	Se positionner sur le niveau précédent
Flèche vers le bas	Se positionner sur le niveau suivant

Q	Quitter l'application
Entrée	Sélectionner le niveau

Voici une explication des touches lors d'un **niveau** :

Touche	Explication
Flèche vers le haut	Effectuer un déplacement vers le haut
Flèche vers le bas	Effectuer un déplacement vers le bas
Flèche vers la gauche	Effectuer un déplacement vers la gauche
Flèche vers la droite	Effectuer un déplacement vers la droite
Q	Quitter l'application
P (Previous)	Undo
N (Next)	Redo

Les images permettant d'afficher à l'écran les objets ont été obtenues en découpant des images disponibles sur Internet via le lien :

https://www.sprisers-resource.com/pc_computer/babaisyou/.

Chaque image correspond à un objet.

On a réussi à mettre en place différentes fonctionnalités du jeu :

- Gestion dynamique des règles (c'est-à-dire qu'elles dépendent de la configuration courante des objets dans la grille).
- Propriétés des personnages prises en compte : YOU, WIN, STOP, PUSH, SINK.
- Undo/Redo.

III - Architecture de l'application

Pour garantir le bon fonctionnement de l'application, il faut mettre en place une architecture robuste.

Quoi de mieux qu'une classe pour construire cette architecture : la classe **App**. Ainsi, l'application est caractérisée par une instance de cette classe.

On souhaite tout d'abord pouvoir lancer l'application. C'est le rôle de la méthode *run()*. Cette est appelée dans la fonction *main()*.

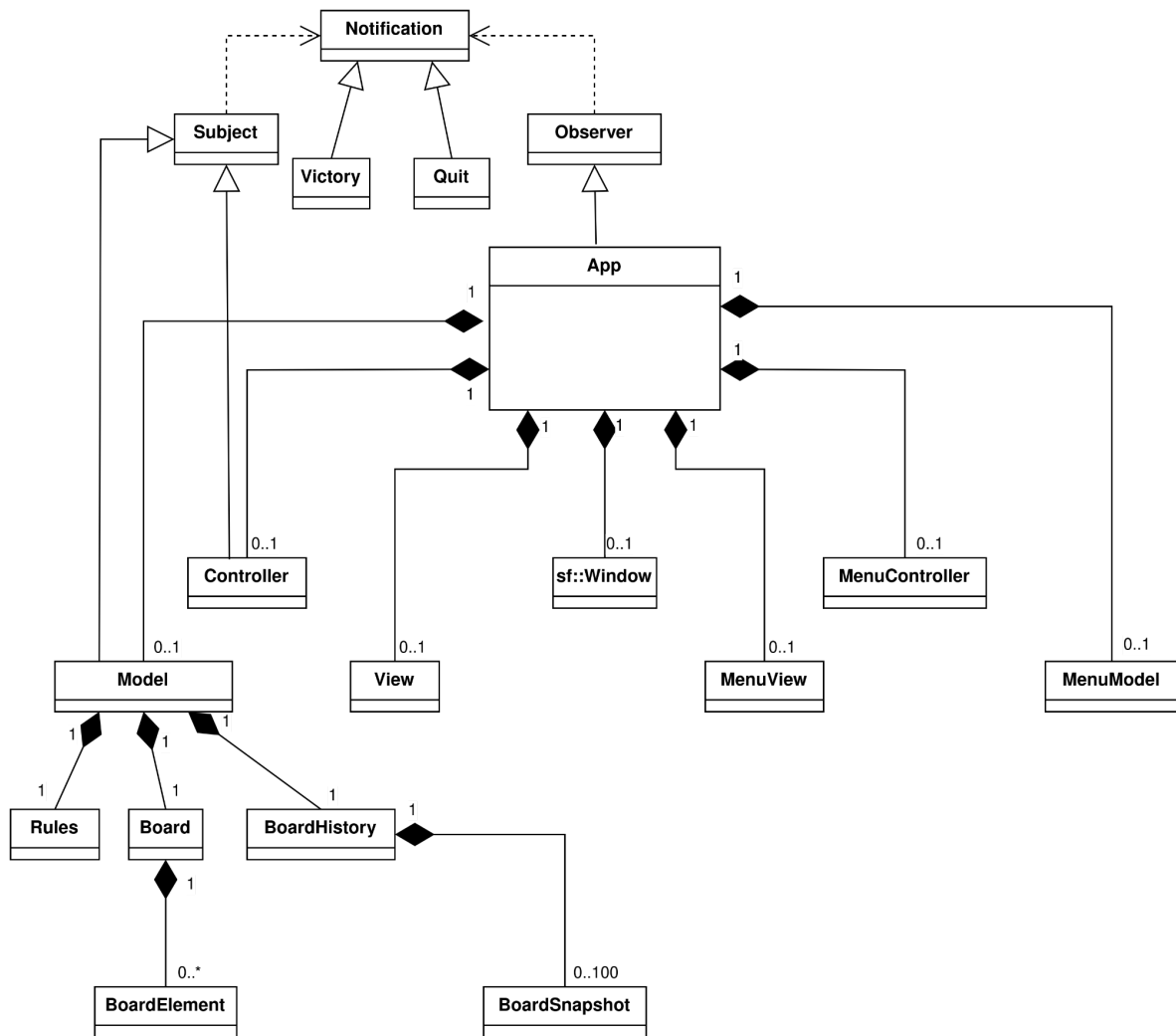
Une fois l'application lancée, une fenêtre s'ouvre. Cette fenêtre est modélisée par un attribut : un pointeur vers un **sf::RenderWindow**. On aurait pu donner comme attribut directement un **sf::RenderWindow** mais on a préféré gérer dynamique la construction et destruction de la fenêtre : la fenêtre est construite lors de la construction de l'application et détruite lors de la destruction de l'application.

Comme dit dans la section précédente, quand l'application tourne, on remarque qu'on se trouve soit dans le menu, soit dans le niveau choisi. On peut interpréter ce comportement de la manière suivante : l'application possède un état qui indique si on se trouve dans le menu ou dans le niveau choisi. Pour modéliser ça, on a décidé d'ajouter l'attribut *appState* de type **AppState**. Les états possibles sont : *AppState::MENU*, *AppState::LEVEL* et *AppState::QUIT* (ce dernier permettant à l'application de se fermer proprement).

Suivant dans quel état se trouve l'application, le menu ou le niveau choisi sera lancé. On peut alors voir chacun d'entre eux comme un sous-processus de l'application. Ça permet de rendre le menu et le lancement d'un niveau indépendant l'un de l'autre. Le sous-processus correspondant au menu est lancé par l'appel de la méthode *processMenu()*, et celui correspondant au niveau choisi est lancé par l'appel de la méthode *processLevel()*. C'est important de comprendre que c'est une alternative : soit c'est le menu qui est lancé, soit c'est le niveau choisi, jamais les deux en même temps !

L'**App**, selon son état, gère la création et la destruction d'un niveau ou du menu. Ces derniers sont implémentés à l'aide du pattern Model - View - Controller (MVC). Initialement, l'**App** crée un menu. Lorsque l'utilisateur désire jouer à un niveau, elle détruit ce menu et crée le niveau. Symétriquement, quand l'utilisateur a gagné, le niveau est détruit et le menu est construit.

Voici l'UML du fonctionnement global de notre application :



On y a fait intervenir seulement les noms des classes pour comprendre comment s'agence l'application. Les explications des classes se trouvant en dessous de **App** seront dans les sections suivantes.

IV - Architecture du menu

Nous avons mis en place un menu assez simple. On ne s'est pas trop attardé dessus pour privilégier le bon fonctionnement d'un niveau. On modélise le menu à l'aide du *pattern* Modèle - Vue - Contrôleur (MVC). On a créé une classe **MenuModel** (pour le modèle), une classe **MenuView** (pour la vue) et une classe **MenuController** (pour le contrôleur). Le menu se traduit par une instance de chacune de ces classes, qui sont liées entre elles. On explique dans les sous-sections suivantes comment chacune de ses classes ont été conçues pour que le menu joue bien son rôle.

A) Modèle du menu

La logique du menu est modélisée par une instance de la classe **MenuModel**.

Le but du menu est de demander à l'utilisateur de faire un choix. Dans notre cas, il doit choisir un des 3 premiers niveaux. Ces 3 choix possibles sont représentés par l'*enum class MenuItem*, dont les valeurs possibles sont *LEVEL1*, *LEVEL2* ou *LEVEL3*.

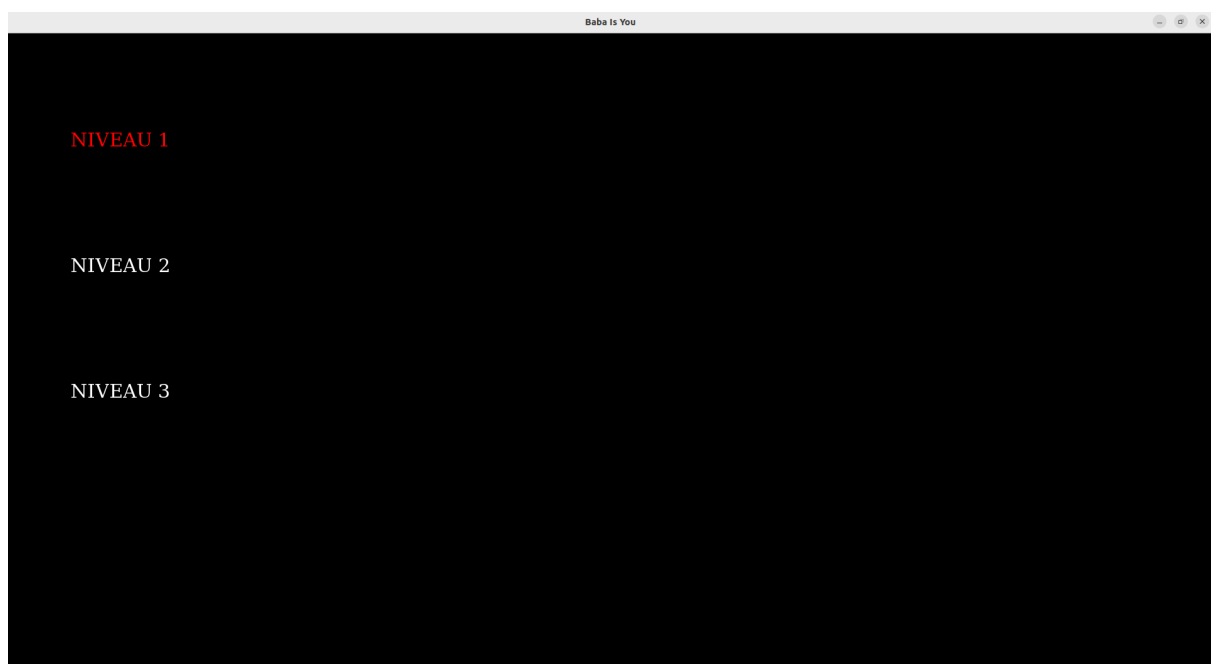
On représente à l'écran ces 3 choix en affichant, les uns au-dessus des autres, les textes *NIVEAU 1*, *NIVEAU 2* et *NIVEAU 3*, dont le fonctionnement de leur affichage sera détaillé dans la prochaine sous-section.

En revanche, l'utilisateur ne fait pas immédiatement un choix. Il peut se positionner sur n'importe quel niveau à n'importe moment jusqu'à qu'il sélectionne le niveau qu'il désire jouer. Pour caractériser le niveau où est positionné l'utilisateur, la classe **MenuModel** possède l'attribut *item* de type **MenuItem**.

Lorsque l'utilisateur décide de se positionner sur le niveau précédent, la méthode *previous()* du modèle est appelée. De même, lorsque l'utilisateur décide de se positionner sur le niveau suivant, la méthode *next()* du modèle est appelée.

B) Vue du menu

Comme dit dans la sous-section précédente, l'utilisateur voit le menu grâce aux 3 textes *NIVEAU 1*, *NIVEAU 2* et *NIVEAU 3* affichés les uns au-dessus des autres dans la fenêtre de l'application. Ces textes sont affichés en blanc sauf celui dont l'utilisateur a décidé de s'y positionner qui est affiché en rouge. Voici un exemple où l'utilisateur s'est positionné sur le niveau 1 :



On modélise la capacité à afficher le menu par une instance de la classe **MenuView**.

Les 3 textes sont représentés dans la classe **MenuView** par les attributs *level1*, *level2* et *level3* tous de type **sf::Text**.

La classe **MenuView** possède comme attribut une référence vers un **sf::RenderWindow**. Elle fait référence à la fenêtre de l'application. Ça permet à la vue d'afficher le menu dans la fenêtre en appelant la méthode *draw()*.

De même, la classe **MenuView** possède comme attribut une référence vers un **MenuModel**. Elle fait référence au modèle de la vue décrit dans la sous-section précédente. Ça permet à la vue de mettre à jour les couleurs des textes d'après le niveau où est positionné l'utilisateur. En effet, l'information concernant le niveau où est positionné l'utilisateur se trouve dans le modèle.

C) Contrôleur du menu

Pour orchestrer le bon fonctionnement du menu, nous avons besoin d'un contrôleur. On rappelle que son rôle est de traduire les intentions de l'utilisateur afin qu'il puisse agir en conséquence en manipulant le modèle et la vue. Comme intentions, il y a : se positionner sur le niveau précédent, se positionner sur le niveau suivant, sélectionner un niveau et quitter l'application. Pour chacune de ces intentions, on associe une touche du clavier (voir II - Vue d'ensemble.).

Le contrôleur est modélisé par une instance de la classe **MenuController**. Les intentions de l'utilisateur se traduisent par des événements dans la fenêtre de l'application. Ils sont traités par la méthode *handleEvent()* de l'instance de **MenuController**. D'où le besoin d'avoir comme attribut dans **MenuController** une référence vers un **sf::RenderWindow** dans **MenuController**, référençant la fenêtre, pour récupérer ses événements.

Si l'utilisateur veut se positionner sur le niveau précédent, le contrôleur va appeler la méthode *previous()* du modèle et la méthode *update()* de la vue. D'où le besoin d'avoir comme attributs dans **MenuController** une référence vers un **MenuModel** (le modèle) et une référence vers un **MenuView** (la vue).

De même, si l'utilisateur veut se positionner sur le niveau suivant, le contrôleur va appeler la méthode *next()* du modèle et la méthode *update()* de la vue.

MenuController a comme attribut *levelRequested* de type *bool*. Il représente un registre qui servira à l'application. Ça lui permet de savoir si l'utilisateur a enfin sélectionné un niveau pour qu'il agit en conséquence (en lançant le niveau). De plus, **MenuController** a comme attribut *whichLevelRequested* de type *int*. Il représente un registre contenant la valeur du niveau sélectionné par l'utilisateur le cas échéant. Ainsi, si l'utilisateur veut sélectionner un niveau, le contrôleur va mettre *levelRequested* à *true* et va mettre la valeur du niveau dans *whichLevelRequested* (1 si niveau 1, 2 si niveau 2 et 3 si niveau 3).

Enfin, **MenuController** possède comme attribut *quitRequested* de type *bool*. Il représente un registre qui servira à l'application. Ça lui permet de savoir si l'utilisateur souhaite quitter l'application pour qu'il agisse en conséquence (en arrêtant l'application). Le cas échéant, le contrôleur met ce registre à *true*.

V - Architecture d'un niveau

A) L'architecture MVC

L'architecture d'un niveau suit un *pattern* Modèle - Vue - Contrôleur (MVC) analogue à celui du menu. Lorsque l'utilisateur désire lancer un niveau, l'**App** à l'aide de sa méthode **initLevel()** crée des références vers les trois composantes du MVC et les lie ensemble. Cette liaison est décrite dans le paragraphe suivant.

Le **Controller** a un rôle assez limité. Il se contente de récupérer les entrées clavier de l'utilisateur et d'appeler les méthodes correspondantes chez le **Model**. Il possède donc une référence vers ce dernier. Le **Model** est le responsable de toute la logique du jeu. Selon les entrées données par l'utilisateur, et reçues grâce au **Controller**, il s'occupe de gérer les modifications du jeu. La **View** observe le **Model** : à chaque mouvement de l'utilisateur, elle observe le **Model** et s'occupe de l'afficher.

C'est l'**App** qui s'occupe ensuite de gérer la boucle principale de jeu : demander au **Controller** de récupérer les *inputs*, nettoyer la fenêtre, dessiner la nouvelle configuration du jeu, et l'afficher. L'**App**, en tant que créatrice de ces objets, en est composée. Elle est donc responsable de leur destruction lors de la fermeture de la fenêtre ou de la fin d'un niveau.

B) La logique *Baba Is You*

Nous avons choisi de modéliser la logique du jeu de la manière suivante. Le **Model** est composé (directement, sans pointeurs cette fois) des **Rules** et du **Board**. Ce sont deux classes indépendantes. En un sens, les **Rules** sont le petit livret qui indique (dynamiquement) les règles du jeu, et le **Board** est le plateau en carton. Le **Model** a une vue d'ensemble sur ces deux éléments (il en est ami), dont il se sert pour gérer l'état du niveau.

Le **Board** (le plateau en carton) est composé (sous forme de pointeurs) des **BoardElement** : ce sont les pions ou, dans le cas de *Baba Is You*, des éléments tant physique (un mur, un Baba ...) que des éléments 'physique avec de la sémantique' (le texte mur, le texte Baba ...). Cette composition se fait sous la forme d'un vecteur 3D, car deux **BoardElement** peuvent se chevaucher. La construction d'un **Board** se fait uniquement à l'aide d'un entier qui indique le fichier texte contenant la configuration initiale du niveau. Une surcharge de l'opérateur () pour **Board** permet de récupérer tous les éléments d'une case. Nous avons choisi d'utiliser des pointeurs pour proposer une composition (la mort du **Board** implique la mort des **BoardElement**) qui évite le danger de copies maladroites lors des déplacements des objets.

La classe **Rules** (le livret du jeu) permet de savoir, à n'importe quelle étape de la partie, si tel élément possède telle propriété. Elle est composée d'une sous classe **RuleKey** qui met en relation un sujet (*e.g* Baba) et une propriété (*e.g* You). Ainsi, l'élément `map<RuleKey, Bool>` permet de dire 'Pour tel sujet et telle propriété, est-ce vrai que le sujet possède cette propriété ?'. Lors de sa construction par le modèle, on initialise toutes les correspondances à faux, en attendant que le modèle calcule immédiatement les règles.

La question de comment modéliser les règles pour les éléments a été l'une des principales difficultés de notre réalisation. Dans cette implémentation, un **BoardElement** ne possède pas "en lui" les règles qui le caractérisent. Ces dernières sont stockées séparément. Le **Model** a accès aux deux et s'occupe d'associer l'objet avec ses propriétés. Nous avons choisi cette implémentation car, *a priori*, intégrer les propriétés directement dans les objets aurait nécessité, à chaque changement de règle, de parcourir l'intégralité du plateau, ce qui est inutilement coûteux dans la mesure où beaucoup d'information est répétée. Une autre idée d'implémentation qui nous est venue *a posteriori* sera discutée dans la Vème partie.

De plus, afin de rendre possible l'Undo et le Redo, nous avons décidé d'utiliser le *pattern* Memento pour stocker l'historique du jeu. Plutôt que de stocker l'intégralité d'un **Board** à chaque étape dans l'optique de le restaurer un jour, l'idée est d'en prendre simplement une "photographie". La classe **BoardSnapshot**, possédant une référence vers le **Board**, s'occupe ainsi lors de sa création de stocker une copie du **Board**, sans se préoccuper de toutes ses autres méthodes. La création d'un **BoardSnapshot** se fait par l'appel d'une méthode sur **Board** : en un sens, le **Board** se prend donc en selfie. La classe **BoardHistory** s'occupe ensuite d'empiler cette photographie dans deux piles, celle destinée à l'Undo et celle destinée au Redo. C'est ensuite au **Model**, lorsque l'*input* de l'utilisateur indique un Undo, d'ajouter la configuration actuelle à la pile Redo, de dépiler la pile Undo, de restaurer le Board, et de recalculer les règles. La surcharge de l'opérateur = pour **Board** et **BoardSnapshot** permet de faire cela très facilement.

C) La gestion d'une partie par le **Model**.

Voyons maintenant comment le **Model** gère concrètement une partie. Comme évoqué, lors de sa construction, il crée un **Board** avec pour paramètre d'entrée le numéro du niveau voulu, une **Rules** où chaque **RuleKey** est initialisée à faux, calcule les règles, et demande au **Board** de se prendre en photo, et le stock dans l'historique.

Pour "calculer les nouvelles règles", le **Model** opère symétriquement en deux temps, d'abord horizontalement puis verticalement. Pour chaque case du jeu, l'objectif est de regarder les cases à côté (horizontalement, puis verticalement) pour essayer de déterminer si nous y reconnaissons la grammaire subject + is + property. Dans ce cas, on ajoute la règle.

Le **Controller**, selon l'*input* de l'utilisateur, peut choisir de :

i) déclencher un déplacement, en utilisant la méthode `move(Direction)` du **Model**.

Le **Model** commence par récupérer tous les éléments ayant la propriété `You`. Il garde en parallèle une liste des éléments qu'il a bougé, pour ne pas le faire deux fois.

On essaye ensuite (*tryMove*) de déplacer chaque élément `You` dans la direction voulue. L'idée est ensuite, récursivement, d'essayer de se déplacer ou de pousser dans cette direction. Si c'est *in fine* possible, on bouge récursivement les éléments. Sinon, on ne fait rien pour cet élément et on passe au suivant. Puis l'on met à jour les règles, on tue les éléments qui se seraient noyés, on prend une photographie du **Board**, et on vérifie si le joueur a gagné.

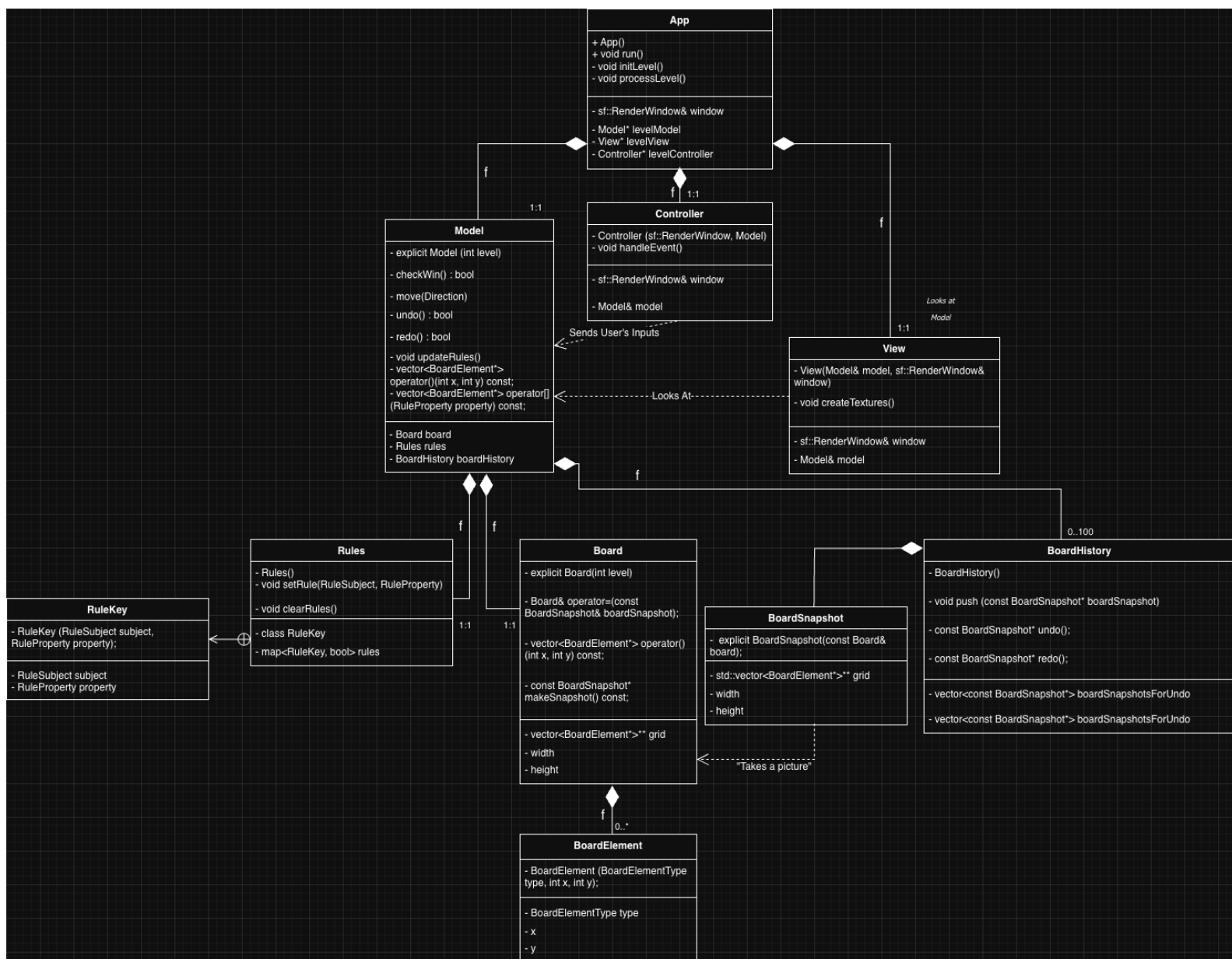
ii) déclencher un *undo* ou un *redo*, en utilisant les méthodes éponymes pour **Model**.

Comme expliqué précédemment, la méthode `undo()` du **Model** appelle la méthode `undo()` de **BoardHistory**, et symétriquement pour le `redo()`.

iii) quitter le jeu, en utilisant ensuite la dynamique développée dans la partie II.

Pour vérifier si l'utilisateur a remporté le niveau, on fait appel à la méthode `checkWin()` du modèle. Elle renvoie *true* si oui, *false* sinon. Cet appel se fait dans la méthode `move(Direction)` du modèle, qui est la seule à changer l'état du plateau dans un état possiblement gagnant pour l'utilisateur. Lorsque le niveau a été remporté, le modèle notifie l'application (via le *pattern* Observateur/Observé) pour qu'elle puisse agir en conséquence : l'application change d'état pour se mettre dans l'état *MENU*. Cette modification d'état appelle la méthode `processMenu()`, qui appelle `initMenu()`, qui se charge de détruire le **Model**, la **Vue** et le **Controller** du niveau. La destruction se fait alors en cascade : par composition, le **Model** détruit automatiquement son `board`, son `boardHistory` et ses `rules`. La destruction du `board` détruit ses `boardElements`, et la destruction du `boardHistory` détruit ses `boardSnapshots`. Bien que cette manière de faire les choses fonctionne, il aurait été plus pertinent de directement détruire le niveau au moment de la victoire, plutôt que de conditionner cette destruction à la création d'un menu consécutif à une victoire.

Comme évoqué, nous avons choisi de restreindre au maximum la disponibilité des constructeurs des classes, afin de nous assurer que seule l'entité responsable est capable de créer et de détruire. Par exemple, le constructeur de **Board** (par ailleurs *explicit* pour éviter les surprises de surcharge) n'est disponible que pour sa classe amie, à savoir **Model**.



VI - Problèmes connus & améliorations envisageables

A) Modélisation de la composition

Notre modélisation de la composition n'est, à certains endroits, pas idéale. Le principe de mettre la classe dominante amie et la quasi-intégralité des méthodes privées permet une certaine sécurité, en interdisant la construction par d'autres entités. Mais cela viole le principe de hiérarchisation. On pourrait vouloir avoir plus de modularité dans les gestions d'accès : typiquement, le **Model** d'un niveau a énormément de pouvoirs, pouvant accéder à tous les attributs des classes qui le composent.

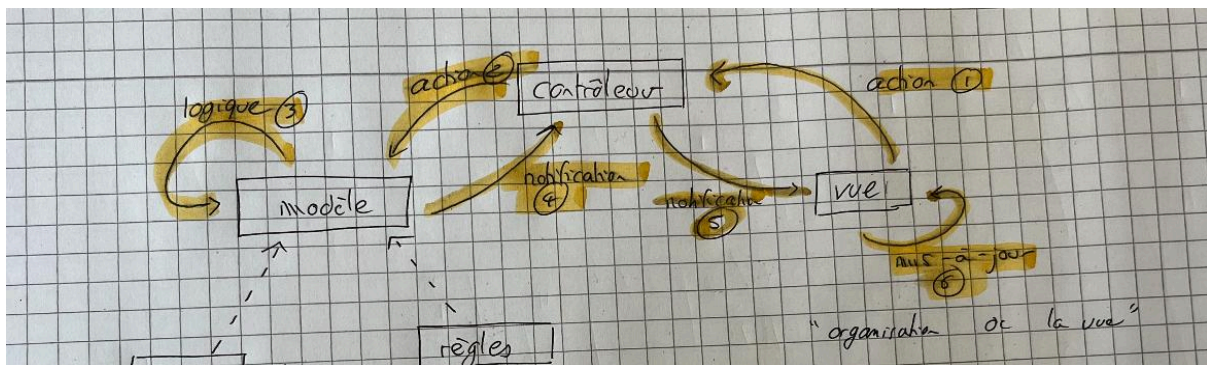
En outre, pour utiliser plus d'héritage nous aurions pu utiliser un héritage privé plutôt qu'une composition : un **Model** aurait ainsi hérité de manière privée de **Rules**, un **Board** et un

BoardHistory. Cette modélisation ne nous a pas semblé naturelle, la mise en place des *friends* permettant de mieux traduire l'idée d'un jeu "tangible" dont le **Model** serait la tête pensante omnisciente.

Pour pousser cette idée, il aurait été pertinent d'implémenter complètement un *pattern* singleton pour les classes **Rules** et **Board**. Dans les faits, une seule instance de ces deux classes est créée par le **Model**, mais l'ajout d'un attribut static contenant l'unique entité aurait ajouté en sécurité.

B) Le MVC

Nous n'étions pas vraiment familiers avec le MVC et avons remarqué que beaucoup de versions très différentes existaient. Dans notre première idée, nous voulions donner beaucoup plus d'importance au **Controller** : renforcer l'indépendance entre le **Model** et la **View** en utilisant le **Controller** comme seul moyen de communication entre ces deux entités. Concrètement, plutôt que la **View** regarde directement le **Model**, c'est le controller qui aurait été chargé de faire transiter le nouvel état du **Model**.



Cette idée nous a semblé trop lourde pour le projet en question, nous avons donc privilégié l'implémentation plus simple évoquée en II. Cela pourrait devenir un problème dans le cas d'une extension du jeu.

C) Construction d'un niveau

Il pourrait être plus efficace d'utiliser un *pattern Prototype* pour la génération de niveau. Plutôt que de détruire un niveau à chaque fois qu'il est gagné, on le garde, et on modifie l'agencement des cases pour qu'il corresponde au nouveau niveau lancé par l'utilisateur.

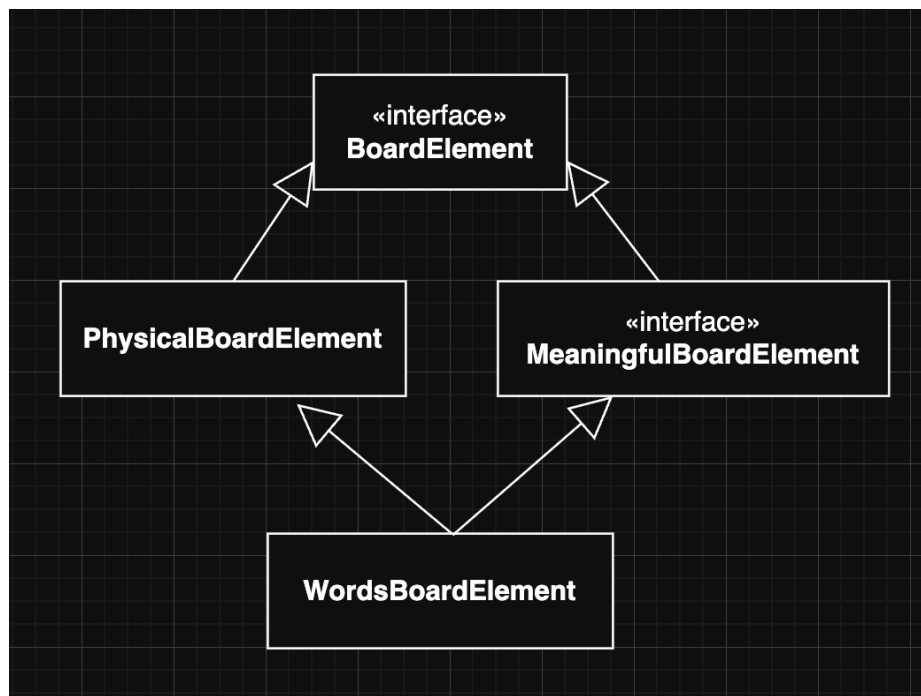
De manière plus anecdotique, notre manière de représenter un niveau est questionnable. Un niveau est stocké sous la forme d'un fichier .txt où la première ligne indique la taille de la grille, puis chaque ligne indique un élément (indexé par un numéro), sa position en x et sa position en y. Si cette représentation permet de représenter facilement des objets qui se superposent, cela rend la construction d'un niveau assez compliquée sans un petit programme python qui permet une traduction automatique.

Dans le menu, les niveaux sont codés “en dur” (niveau 1, niveau 2, etc ...). Pour des développements futurs, on pourrait imaginer un moyen de déterminer le nombre d’éléments à afficher en fonction du nombre de fichiers .txt présents.

Enfin, une méthode traductrice permet de convertir le numéro inscrit dans le fichier .txt du niveau entre l’entité correspondante. En réalité, un `static_cast<int>` depuis l’enum class contenant ces entités aurait suffi.

D) Optimisation et modélisation des **BoardElements**

Dans une première implémentation, nous avons choisis d’implémenter les **BoardElements** avec de l’héritage multiple :



L’idée étant qu’un **BoardElement** peut être ou bien physique (un mur, un baba ...), ou bien avoir une dimension sémantique. Mais comme, dans *Baba Is You*, les règles (au sens de text baba, text mur ...) sont aussi des éléments du jeu, elles sont aussi des éléments physiques. Des `dynamic_cast` permettraient ensuite de déterminer facilement le type d’un élément. Cette solution a été abandonnée car bien que moins “élégante”, la simple utilisation d’un champ type permettait de faire cela beaucoup plus facilement.

D’autre part, notamment pour la méthode `sink()`, on aurait pu imaginer, plutôt que de détruire l’élément, de simplement le “cacher” en dehors du plateau. Si cette solution est plus économe, la “vraie” destruction d’un élément nous permettait d’illustrer l’efficacité de notre *pattern* Memento.

E) Pattern Memento

De petites améliorations pourraient être faites sur le *pattern* Memento. Plutôt que de stocker des vecteurs, il aurait été mieux de stocker des *stacks*. Plutôt que de stocker deux piles (une pour l'undo, une pour le redo), on aurait pu imaginer que l'historique de la partie est stockée sous la forme d'un "ruban de lecture" qui se déplace vers la gauche lors d'un undo, et vers la droite lors d'un redo.

De plus, il semble *a priori* bien plus efficace de ne pas à chaque coût prendre en photo tout le **Board**, mais simplement les éléments qui ont été modifiés. Mais encore une fois, au vu de la taille du plateau, notre solution est viable.

F) Modélisation des règles

Comme évoqué dans la partie IV, nous avons pensé *a priori* à deux manières de modéliser les règles : ou bien directement dans les objets (*i.e* chaque **BoardElement** a pour attribut les règles qu'il possède), ou bien de manière externe. Nous avons choisi la seconde option, qui permettait d'économiser des répétitions et des parcours de la grille pour mettre à jour les éléments. Cette implémentation a cependant l'inconvénient de nous forcer à tout de même devoir parcourir la grille, notamment pour vérifier la condition de victoire (*i.e* qu'un élément YOU se trouve sur la même case qu'un élément WIN).

L'un des problèmes de notre implémentation actuelle est la rigidité de notre grammaire : l'obligation qu'une règle soit composée d'un sujet et d'une propriété rend difficile la gestion des règles du type "Baba is Wall".

Une autre idée, qui nous est venue en discutant avec nos camarades, serait d'utiliser un modèle flyweight pour obtenir le "meilleur des deux mondes". Le **Board**, plutôt que d'encapsuler un tableau 3D de pointeurs vers des **BoardElements** différents, encapsulerait des pointeurs vers un seul et unique élément de leur type. Par exemple, plutôt que d'avoir 10 murs codés par 10 BoardElement différents, il s'agirait d'un seul et même objet qu'on affichait à 10 endroits différents. Ce qui rendrait la gestion des règles bien plus simple.

G) Le Model

Dans notre implémentation, le **Model** a beaucoup de responsabilités. S'il fallait lui ajouter des fonctionnalités (e.g pour gérer la lave), il serait bon de le séparer en plusieurs sous classes pour conserver le principe de modularité.

La méthode Move() fonctionne actuellement de manière récursive, mais nous aurions pu envisager d'utiliser le *pattern Chain of Responsibility* pour faire une implémentation similaire.